AI Computational Commons

Independent efficiency research synthesis

Research status: Open; economic and native-language hypotheses not yet validated
Prepared: November 4, 2026
Concept and architecture: Izik Bachar
Reviewed inputs: Claude Opus 5.5, Gemini 3 Flash, Grok 4; prior GPT-6 Round 7 / Round 8 audit
This document: A critical synthesis of independently supplied reports, not a new production benchmark. The main article remains open at Draft 4.

Executive decision

Continue research; do not claim proven token, dollar, or end-to-end time savings. All three reports agree that software capability reuse is feasible and the symbolic-language hypothesis has not been tested with a model trained on dedicated native operation tokens. Claude adds reproducible *host execution* microbenchmarks, but no report contains provider-billed token telemetry, full task charges, or model inference latency from a randomized multi-session A/B study. The largest missing piece is **incremental value over strong conventional agents already equipped with libraries, tool search, code execution, and caching**.

1. Evidence by metric

Metric	Observed in supplied reports	What cannot be concluded
Task correctness	R8 conventional and registry solutions reached 20/20; prior R7 14/14 for reuse, 6/6 for both controller authoring arms	Superior reliability or economics of AICC at scale
Static tokenizer count	Claude: R8 Python solution **528** o200k tokens, AICC version **713** (+35%); symbolic controller **29** tokens versus **27** for minimal Python with equal host-provided templates	Actual inference token usage, billed cost, or trained-native-token performance
Provider token billing	**Not measured** by any agent	Real input, cached-input, output, or reasoning-token savings
Runtime CPU latency	Claude measured Python mock-binding and symbolic execution overhead; independently spot-checked one binding pattern here	Whole-agent turnaround speed or production sandbox performance
Total dollar cost	**Not measured**; some reports contain cost models or predictions	ROI, break-even reuse count, or enterprise savings
Private-code security	Simple mock scope and revocation checks succeed (3/3)	Enforced sandbox isolation, encryption, or secure multi-agent federation

Measurement integrity: Static tokenization of generated source is not the same as API usage. Executing finished code is not a measurement of the time or cost an AI took to *create* that code. A mock that checks declared effects does not enforce process-level security.

2. Claude: the strongest new experimental contribution

Claude supplied source scripts, raw measurements, simulation code, trial logs, and a measurement CSV. The following are **environment-specific** observations (Python/Node sandbox, one vCPU); they are not production guarantees.

Microbenchmark	Direct / conventional	Experimental AICC or symbolic	Interpretation
`normalize_tag`, median per call	**0.26 µs** direct	**14.97 µs** revalidated AICC mock	Approximately 58× on a trivial hot-path operation, largely due to per-call file hashing
`group_by_window` on 50 events	**72.2 µs** direct	**89.1 µs** AICC mock	Approximately 23% slower in this fixture
Persistent worker IPC	**0.26 µs** in process	**13.0 µs** through pipes	Boundary overhead can matter for granular operations
New process per call	**0.26 µs** in process	**34 ms**	Avoid process startup on every capability invocation
Controller, n=1000	**185 µs** native Python	**205 µs** symbolic compiled to Python	Compilation can approach native-host execution cost
Controller, n=1000	**185 µs** native Python	**950–4,284 µs** toy interpreters	Naive interpretation can be several-fold to ~23× slower

Independent spot check: Re-executing the supplied Round 8 runtime on this review environment produced approximately **0.20 µs** for a direct `normalize_tag` call and **14.03 µs** through the mock AICC binding. The pattern is consistent with Claude's finding; differences between machines prohibit treating values as interchangeable.

Claude also ran ten **fresh-process harness executions** per arm, with both R8 solutions passing 20/20. These are repeated runs of already-written code, **not** independent AI authoring trials. Its simulation (not measured task costs) suggests small studies may have little power to detect a true 15% cost difference under high stochastic variance. Sample size must be based on an instrumented pilot, not accepted from a hypothetical distribution alone.

3. Gemini: useful agreement, important unsupported claims

Gemini agrees that AICC has not demonstrated economic superiority against conventional cached libraries. However, several precise claims in its submission are not supported by reproducible experiment records:

- An asserted **14.2% increase in syntax errors** for symbolic programs lacks a displayed trial design, denominator, or raw error observations.

- A proposed **10-execution break-even threshold** lacks demonstrated setup, maintenance, or per-use dollar costs.
- A reported **0.15–0.50 ms per-binding overhead** lacks raw timing logs and differs from Claude's specific Python-mock benchmarks. This does not mean the figure is impossible; it is simply **unverified**.
- The claim that a combined AICC + symbolic implementation passed the 20-case R8 application suite is not backed by a complete executable implementation showing the toy grammar handling those workflows. The supplied grammar was not general-purpose.

Treat these as **claims requiring additional evidence**, not measured AICC results. Gemini's broad conclusion—that a dedicated operation-token model remains untested—is appropriate.

4. Grok: conservative evidence accounting

Grok re-executed the official R8 workflow tests and returned both conventional and AICC solutions plus a measurement template. I independently reran its submitted implementations against the frozen packet: **20/20 for A, 20/20 for B, and 3/3 mock authorization checks**. Grok did not claim API billing, end-to-end speed, model-native token, or all-in dollar measurements. Its principal caution is that existing prompt caching, deferred tool loading, and documented package APIs already capture much of the obvious reuse advantage.

5. Externally verified benchmarks: motivation, not AICC performance

- **Ghoshal et al., EMNLP 2026**, studied an operation-visible enterprise API agent with a deterministic operation resolver. The reported system used **54.1% fewer tokens**, had **43.3% lower median latency**, and improved task success from **68% to 77%** versus its production-derived baseline. It was **not** AICC and did not evaluate the proposed symbolic language. ACL Anthology
- **Liu et al., CompactionBench, Microsoft Research, Oct 2026**, reported up to **25.4% lower total cost** and **9.6% less agent time** using asynchronous progressive context compaction. Again, this was **not** AICC. Microsoft Research

- **Bai et al., April 2026**, found that coding-agent runs on the same task could vary by **up to 30×** in total token usage. Repeated, randomized trials are therefore essential. Stanford Digital Economy Lab
- **Wu, Anderson and Guha, 2026** documented higher token expenditure and repair loops in less familiar coding languages. This is a direct warning against inferring model efficiency from dense symbolic syntax alone. arXiv

6. Updated hypothesis assessment

Hypothesis	Status after new reports	Decisive missing evidence
H1: Public computational commons	Reuse feasibility demonstrated; **incremental advantage unproven**	Lower billed *cost per correct task* than normal well-documented packages with search and caching
H2: AI-native language	Toy representation executable; **trained native-token hypothesis untested**	Model with native operation tokens or structured action vocabulary tested on unfamiliar programs against equally equipped conventional model
H3: Persistent validated bindings	Host-side mechanism plausible; **economic gain and robust revocation unproven**	Multi-session persistence with real invalidation and authorization; cost versus equally optimized ordinary caching
H4: Public/private federation	Architectural proposal only	Real isolation, cryptographic key custody, artifact identity, permission revocation, and cross-agent contract conformance

7. The required next experiment

First, isolate the commons economics. Use the same model, existing library implementations, hidden tests, and execution hardware in both arms. Arm A uses ordinary modules, package docs, tool search, code execution and prompt caching. Arm B uses the identical functions through an AICC registry with verified versioned handles. Test unfamiliar multi-file tasks involving concurrency, scoped memory, transactions, event pipelines, image processing, and authorization boundaries.

Measure every trial: uncached input tokens, cached input tokens, output/reasoning tokens; billed dollars; tool and model wall time; calls and retries; CPU seconds and peak RSS; failed test outcomes; setup and maintenance costs amortized over actual reuse; cold and warm

paths separately. Require correctness and security non-inferiority. Randomize, use independent model sessions, and compute confidence intervals. A pilot must determine the sample size.

Separately, test the language hypothesis. A textual punctuation DSL under today's tokenizer is not a valid stand-in for a trained native-operation model. Compare conventional authoring with a typed operation representation at matched tool access, then—in a properly resourced separate study—a model specifically adapted to atomic operations. Evaluate correctness, syntax failures, repairs, inference tokens and compile/runtime time. The grammar in our article remains **illustrative theory, not a final design decision**.

Break-even analysis: A commons is economically favorable only when cumulative *net* savings exceed initial implementation, contract curation, verification, security review and ongoing registry maintenance. Any claim such as "10 uses breaks even" is meaningless without measured costs and a stated deployment volume.

Publication rule

The proposed AICC architecture may be recommended as a **research direction**, not as a proven cheaper or faster replacement for existing software development. The current article should remain open. Preserve the separate claims: (1) reuse established capabilities; (2) investigate a genuinely AI-native language for *new* programming logic. Neither claim should inherit the published savings of unrelated tool-optimization systems.

Evidence provenance: Claude `AICC_Efficiency_Research_Report_Claude.md` and nested code/log package; Grok `Grok_AICC_Efficiency_Independent_Research_Report.md` and included submissions; Gemini `aicc_efficiency_independent_research_report (1).md`; Round 7 and Round 8 frozen packets. The three external quantitative studies above were checked against their publisher or institutional pages on November 4, 2026. No paid API consumption telemetry was available from the supplied experiment reports.